



# Inside Laravel

The patterns that power  
the framework

---

Mohammad Emran Hasan  
Ahmed Shamim Hasan Shaon

# Inside Laravel: The Patterns That Power the Framework

Copyright © 2026 Mohammad Emran Hasan and Ahmed Shamim Hasan Shaon.

All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the authors, except for brief quotations used in reviews and certain other noncommercial uses permitted by copyright law.

This book is provided for informational and educational purposes only. The authors make no warranties as to the accuracy or completeness of its contents and disclaim any liability for losses or damages arising from its use. Code examples are provided "as is," without warranty of any kind.

Laravel is a trademark of Taylor Otwell. This book is not affiliated with, endorsed by, or sponsored by Laravel or Taylor Otwell. All other trademarks are the property of their respective owners and are used for identification purposes only.

Published by the authors.

Cover design by [Osama Adnan](#)

First edition, 2026. Version 1.0

[insidelaravelbook.com](https://insidelaravelbook.com)

## Why this book exists

Laravel documentation is excellent for learning *how* to use features. But it doesn't explain *why* similar ideas keep appearing in different places, or how recognizing those ideas can make you a better developer.

This book fills that gap.

## Who this book is for

This book assumes you already know Laravel. You've built applications, used queues and caching, worked with the HTTP client, and probably debugged a few production issues along the way.

If you're still learning the basics (routing, controllers, Eloquent, Blade), you'll get more from this book after you've spent some time building real things.

## What this book covers

This book focuses on the patterns that show up in multiple places across Laravel, the recurring concepts that hold the framework together.

We'll explore:

- Retry logic
- Failover mechanisms
- Atomic locks

- Rate limiting
- Pipelines
- Events & observers
- Store/driver abstraction
- Serialization & transport
- Middleware
- Macroable / runtime extensibility

For each one, you'll see where it appears across Laravel and why it's used there.

## What this book doesn't cover

- How to build a full Laravel application
- Eloquent fundamentals
- Blade, Livewire, Vue, or frontend topics
- General architecture theory or design patterns not used in Laravel
- Internal framework source-code deep dives

The goal is to stay focused on connecting the dots between reusable ideas.

If you want to learn about these excluded topics, please head to the excellent resources at [Laracasts](#) and [Laravel Daily](#), they are the OGs.

## **How to read this book**

You don't need to read this book in one sitting. Each chapter stands on its own, but you'll get more out of reading them in order since many concepts build on each other.

As you read, try connecting what you see to your own projects. Where have you run into this pattern before? Could you use this idea in your own packages? That's where the real value is.

# Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| Introduction.....                                              | 8  |
| Chapter 1: The Idea of Cross-Cutting Concepts in Laravel ..... | 10 |
| Why cross-cutting patterns matter.....                         | 10 |
| You learn new features faster.....                             | 10 |
| Your own code becomes more "Laravel-like" .....                | 11 |
| You become a better debugger.....                              | 11 |
| Patterns vs. classical design patterns.....                    | 11 |
| Laravel's design philosophy .....                              | 12 |
| Seeing Laravel differently.....                                | 12 |
| Chapter 2: The Retry Pattern.....                              | 14 |
| Why Retries Matter .....                                       | 15 |
| Database Transactions.....                                     | 15 |
| HTTP Requests.....                                             | 16 |
| Queued Jobs.....                                               | 17 |
| Retrying Failed Jobs Manually.....                             | 18 |
| The retry() Helper .....                                       | 19 |
| Things to Keep in Mind .....                                   | 19 |
| The Pattern Across Laravel .....                               | 20 |
| Chapter 3: Failover Mechanism.....                             | 21 |
| Failover in Queues .....                                       | 22 |
| Failover in Cache.....                                         | 24 |
| Failover in Mail.....                                          | 27 |
| Why Failover Is a Cross-Cutting Pattern .....                  | 29 |
| Best Practices .....                                           | 29 |
| How Failover Differs from Retry.....                           | 29 |
| Chapter 4: Atomic Locks & Concurrency Control.....             | 31 |
| What an Atomic Lock Is.....                                    | 32 |
| Where Laravel Uses Locks .....                                 | 33 |
| Preventing Overlapping Queue Jobs.....                         | 33 |
| Unique Jobs: Locking at Dispatch Time .....                    | 34 |
| Task Scheduling .....                                          | 34 |
| Isolatable Artisan Commands .....                              | 35 |
| Isolated Migrations.....                                       | 35 |

|                                                  |    |
|--------------------------------------------------|----|
| Session Blocking . . . . .                       | 35 |
| Considerations When Using Locks . . . . .        | 36 |
| The Pattern Across Laravel . . . . .             | 37 |
| Chapter 5: Rate Limiting & Throttling . . . . .  | 38 |
| Why Rate Limiting Exists . . . . .               | 38 |
| The RateLimiter Abstraction . . . . .            | 39 |
| HTTP Request Throttling . . . . .                | 40 |
| Exception Throttling . . . . .                   | 40 |
| Throttling Queue Jobs . . . . .                  | 41 |
| Why Redis Matters for Throttling . . . . .       | 42 |
| Choosing the Right Scope . . . . .               | 43 |
| Using Rate Limits Beyond HTTP . . . . .          | 44 |
| Things to Keep in Mind . . . . .                 | 45 |
| The Pattern Across Laravel . . . . .             | 45 |
| Chapter 6: The Pipeline Pattern . . . . .        | 46 |
| How Pipelines Work . . . . .                     | 47 |
| A Simple Example . . . . .                       | 47 |
| HTTP Middleware . . . . .                        | 48 |
| Job Middleware . . . . .                         | 49 |
| HTTP Client Middleware . . . . .                 | 49 |
| Process Pipelines . . . . .                      | 50 |
| The Pipeline Class . . . . .                     | 51 |
| When to Use Pipelines in Your Own Code . . . . . | 51 |
| The Pattern Across Laravel . . . . .             | 52 |
| Chapter 7: Events & Observers . . . . .          | 53 |
| What Events Solve . . . . .                      | 54 |
| Using Events in Your Own Code . . . . .          | 55 |
| How Laravel Itself Uses This Pattern . . . . .   | 56 |
| Queue Events . . . . .                           | 56 |
| Queued Listeners . . . . .                       | 58 |
| When to Use Events, and When Not To . . . . .    | 58 |
| The Pattern Across Laravel . . . . .             | 59 |
| Chapter 8: Driver Pattern . . . . .              | 60 |
| How Laravel Structures This . . . . .            | 61 |
| Where This Pattern Appears . . . . .             | 62 |
| Why This Pattern Matters . . . . .               | 64 |
| The Terminology Varies . . . . .                 | 65 |
| Applying This Pattern in Your Own Code . . . . . | 65 |

|                                     |    |
|-------------------------------------|----|
| The Pattern Across Laravel .....    | 66 |
| Chapter 9: Deferring Work .....     | 67 |
| After-Response Execution .....      | 68 |
| Deferring Events .....              | 69 |
| Deferring HTTP Client Batches ..... | 70 |
| Deferred Service Providers .....    | 70 |
| Defer vs. Queue .....               | 71 |
| The Pattern Across Laravel .....    | 72 |
| Conclusion .....                    | 73 |
| About the Authors .....             | 75 |

# Introduction

There's a moment most Laravel developers experience at some point.

You're working on something, maybe adding retry logic to an HTTP call, or figuring out how to prevent two queue jobs from running at the same time, and you realize you've seen this before.

Not in this exact place, but somewhere else in the framework. The shape of the solution feels familiar. The method names remind you of something you used last month in a completely different context.

You start to wonder: is Laravel reusing the same ideas across different features?

The answer is yes.

And once you see it, you can't unsee it.

Laravel is built around a small set of recurring concepts that show up throughout the framework in different forms. The same ideas that power one feature quietly power a dozen others. This isn't accidental. It's a deliberate design choice that makes the framework feel coherent even as it grows.

This book is about those recurring concepts.

If you've built real applications with Laravel, shipped features, debugged production issues, wrestled with queues, caching, or notifications, then you've already touched these patterns. You probably didn't recognize them at the time. That's fine. Most people don't.

Understanding what they are and where they appear will change how you work with the framework. You'll reach for the right solution faster. You'll write code that fits with Laravel instead of fighting it.

This isn't a source-code deep dive, and it's not trying to catalog every feature. It focuses on the handful of ideas Laravel uses repeatedly and shows how each one appears across different subsystems.

Think of it as sitting down with a teammate who's been working with Laravel for years and asking, "Hey, why does Laravel do it this way?" and getting a straight answer.

By the end, you'll notice things you didn't notice before. Features that once felt separate will start to feel related. The next time Laravel releases something new, you'll probably recognize the concept behind it before you finish reading the announcement.

Let's get started.

# Chapter 1: The Idea of Cross-Cutting Concepts in Laravel

The Introduction gave you a glimpse of what we're talking about: recurring ideas that appear throughout Laravel in different forms. This chapter explains why that matters, and how to think about what you're seeing.

## Why cross-cutting patterns matter

Most developers learn Laravel feature by feature:

- How to send a request
- How to dispatch a job
- How to cache something
- How to send a notification

This is natural. It's how tutorials are written, how documentation is organized, how most of us learn anything new. One piece at a time.

But beneath each feature, the same ideas appear again and again. Recognizing them helps you in a few concrete ways.

## You learn new features faster

When a new Laravel feature lands (job middleware, HTTP client middleware, bus batches), you can understand it immediately if it uses a pattern you already know. You're not starting from scratch. You're applying something familiar.

This compounds over time. The more patterns you recognize, the faster every new feature clicks.

## **Your own code becomes more "Laravel-like"**

Once you see how Laravel solves problems, you naturally reach for the same tools: writing your own pipelines, using events to decouple systems, adding driver-based architecture to your packages, etc.

Other Laravel developers can read your code and immediately understand what you're doing, because you're speaking the same language.

## **You become a better debugger**

When something breaks, knowing the concepts behind the scenes changes how you investigate. Instead of guessing, you can predict where the problem likely lives and trace issues across subsystems that share the same underlying idea.

Debugging becomes pattern recognition.

## **Patterns vs. classical design patterns**

A quick clarification before we go further.

This book isn't about formal "Design Patterns" like Strategy, Observer, or Factory, the kind you'd find in a Gang of Four textbook. Laravel uses some of those, but it doesn't follow them rigidly. It adapts ideas into practical forms that fit modern PHP applications.

For example:

- Laravel's pipeline is inspired by Chain of Responsibility, but it's not a textbook implementation.
- Laravel's retry logic resembles the Retry Pattern from distributed systems, but it's simpler and more pragmatic.
- Laravel's drivers solve problems similar to the Adapter pattern, but you'd never confuse them.

When this book says "pattern," it means something practical:

*A reusable concept that Laravel applies across different features to solve similar problems.*

If you're familiar with classical design patterns, you'll notice echoes of them. If you're not, that's fine. You don't need that background to follow along.

## **Laravel's design philosophy**

These patterns don't appear by accident. They come from a consistent design philosophy.

Laravel favors code that reads naturally. When you see `$response->retry(3)` or `Cache::lock('key')->get()`, you understand what's happening without checking the docs. This is why the same concepts can appear in different contexts without feeling foreign.

Laravel also deliberately reuses patterns so that knowledge transfers across features. When retry logic works the same way in the HTTP client, queue jobs, and database transactions, you only learn the concept once. Your mental model stays simple even as you use more of the framework.

And Laravel is pragmatic about it. It doesn't implement patterns for their own sake. If a textbook pattern needs modification to be practical, Laravel modifies it. If a simpler approach works just as well, it uses the simpler approach.

Many of these patterns (drivers, middleware, macros) are designed for you to extend. The framework exposes them so you can plug in your own implementations. Understanding the patterns means understanding how to build on top of them.

## **Seeing Laravel differently**

Once you internalize these cross-cutting concepts, things start to click. Features that once felt separate start to feel related. You'll read about something new in the changelog and think, "Oh, that's the pipeline pattern again." You'll encounter a bug and immediately know which concept is involved.

You don't need to memorize framework internals. You just need to recognize the handful of ideas that Laravel returns to again and again.

That's what the rest of this book is about.

On to the first pattern.

## Chapter 2: The Retry Pattern

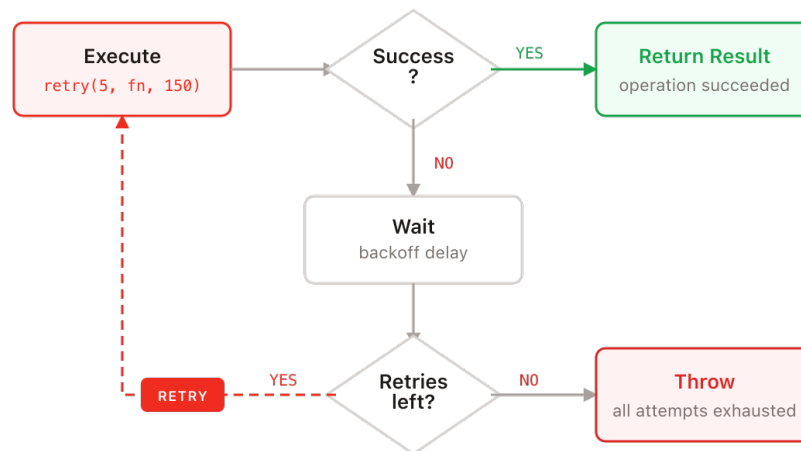
You're integrating with a payment gateway. Everything works in development, but in production, the API occasionally times out. Not often, maybe once every few hundred requests, but enough that users see errors. The fix isn't to make the API more reliable (you can't control that). The fix is to 'try again'.

This is the kind of problem retry logic solves. If you've worked with Laravel for any length of time, you've probably already used it, maybe without realizing how deeply the idea runs through the framework.

Retry isn't a feature that lives in one place. It appears in database transactions, HTTP requests, queued jobs, and even as a standalone helper you can wrap around any code. The API looks slightly different in each context, but the concept is always the same: if something fails temporarily, try again before giving up.

### The Retry Pattern

*If something fails for a temporary reason, try again before giving up*



## Why Retries Matter

**Transient failures** are temporary issues: a network glitch, a momentary service outage, a deadlock in the database. These aren't bugs in your logic. They're environmental. And often, simply trying again resolves the issue.

Without retry logic, these small failures can cascade into bigger problems. A single API timeout breaks a user's checkout flow. A queued job fails due to a brief network hiccup and never runs again. A database deadlock aborts a transaction that would have succeeded a moment later.

Can retries solve every failure?

Not really.

The key distinction is **transient vs. permanent failures**. A network timeout is transient: the server was briefly unreachable, and it'll probably work in a second. A validation error or wrong credentials? That's permanent. Retrying won't fix bad data or a wrong password. This distinction matters because retrying a permanent failure wastes time and obscures the real problem.

Laravel's retry abstractions help you handle transient failures in a clean, expressive way, and they work the same way across different subsystems.

## Database Transactions

Database deadlocks are a classic case for retries. Two queries lock rows in conflicting order, one gets killed, and the transaction fails even though nothing was actually wrong with your code.

Laravel's `DB::transaction()` accepts a second argument: **the number of attempts**.

```
DB::transaction(function () {  
    DB::table('users')->where('id', 1)->update(['votes' => 1]);  
    DB::table('posts')->where('user_id', 1)->delete();  
}, 5);
```

If a deadlock occurs, Laravel catches the exception and reruns the closure, up to 5 times. The default is 1 (no retries), so if you're working with tables that see concurrent writes, passing an explicit count is worth doing.

```
// The method signature; default is 1 attempt (no retries)
public function transaction(Closure $callback, $attempts = 1) {...}
```

This only catches deadlock-type exceptions, not every possible failure. A query syntax error won't be retried, which is correct. Retrying a permanent failure would just fail again.

**Tip:** Always pass an explicit attempt count instead of relying on the default. It makes your intent clear to anyone reading the code.

Ref: <https://laravel.com/docs/master/database#handling-deadlocks>

## HTTP Requests

External APIs are unreliable by nature. Timeouts happen, rate limits kick in, and servers go down briefly. Laravel's HTTP client has retry built in.

```
$response = Http::retry(3, 100)->post('https://api.example.com/data', [
    'name' => 'Alice',
    'role' => 'admin',
]);
```

This allows up to 3 attempts, with a 100-millisecond wait between each. The delay matters. If a server is momentarily overloaded, hitting it again immediately usually makes things worse.

You can pass a closure that decides whether a given failure is worth retrying:

```
$response = Http::retry(5, 200, function ($exception, $attempt) {  
    return $exception instanceof ConnectionException  
        || $exception instanceof RequestException;  
})->get('https://api.example.com/resource');
```

The closure receives the exception and the current attempt number. Return `true` to retry, `false` to stop. This is important because not every HTTP failure is transient. A `422 Unprocessable Entity` means your request data is wrong, and retrying won't fix that.

Ref: <https://laravel.com/docs/master/http-client#retries>

## Queued Jobs

Jobs fail. A downstream service might be temporarily unavailable, a rate limit might be hit, or a network connection might drop. Laravel gives you several ways to control retry behavior on a per-job basis.

```
class ProcessOrder implements ShouldQueue  
{  
    use Queueable, InteractsWithQueue, SerializesModels;  
  
    public $tries = 5;  
    public $backoff = 10;  
  
    public function handle() {} // Process the order!  
  
    public function failed(Throwable $e)  
    {  
        Log::error('Order processing failed permanently: ' . $e->getMessage());  
    }  
}
```

`$tries` sets the maximum number of attempts. `$backoff` sets the delay in seconds between them. If all attempts are exhausted, the `failed()` method runs. This is your chance to log the problem, notify someone, or clean up.

Sometimes you don't know how many attempts you'll need, but you know how long you're willing to wait. That's what `retryUntil()` is for:

```
public function retryUntil()
{
    return now()->addMinutes(10);
}
```

Laravel will keep retrying the job until the timestamp passes, regardless of how many attempts it takes. This is useful for jobs that depend on external systems with unpredictable recovery times.

You can also define `tries()` and `backoff()` as methods instead of properties when you need dynamic logic, for example, different backoff intervals for each attempt.

You can also control retry behavior globally when starting a worker:

```
php artisan queue:work --tries=3
```

This sets the maximum attempts for all jobs processed by that worker. Per-job properties override this when defined.

Ref: <https://laravel.com/docs/master/queues#max-job-attempts-and-timeout>

## Retrying Failed Jobs Manually

Sometimes jobs exhaust all their retries and fail permanently. Laravel records these in the `failed_jobs` table, and you can retry them with Artisan commands:

```
php artisan queue:retry {id}          # retry a specific job
php artisan queue:retry --queue=emails # retry all failed jobs on a queue
```

```
php artisan queue:retry all
```

```
# retry everything
```

This is useful when the transient issue has been resolved (a service has come back online, a config has been fixed) and you want to reprocess jobs that failed during the outage.

Ref: <https://laravel.com/docs/master/queues#retrying-failed-jobs>

## The `retry()` Helper

Everything above is specific to a subsystem: transactions, HTTP, queues, etc. But what if you need retry logic somewhere else entirely?

Laravel provides a `retry()` helper that works anywhere:

```
$result = retry(5, function () {  
    return SomeExternalService::fetch('resource-id');  
}, 150);
```

Five attempts, 150 milliseconds between each. If the closure succeeds, you get the return value. If all attempts fail, the last exception is rethrown.

This is the most general form of the pattern. Any code that might fail transiently (e.g., a file lock, a cache write, or a third-party SDK call) can be wrapped in `retry()` without requiring framework-level support for that specific operation.

Ref: <https://laravel.com/docs/master/helpers#method-retry>

## Things to Keep in Mind

Retry logic needs to be applied deliberately.

**Know what's transient and what isn't.** A network timeout is transient. A validation error is not. Retrying a permanent failure wastes time and obscures the real problem. In HTTP contexts, use the closure form to filter which exceptions are worth retrying. In jobs, consider whether the failure type actually benefits from another attempt.

If a service is struggling under load, retrying immediately adds to the pressure. Even a small delay gives the target time to recover. For high-traffic systems, consider **exponential backoff**, which Laravel's queue jobs support via an array of backoff values.

**Set limits you can reason about.** Five retries with a 200ms delay mean your request might take an extra second. Five retries with a 30-second backoff on a queued job could mean the job runs for several minutes. Make sure the retry window makes sense for the user experience or system behavior you're designing.

When all retries are exhausted, you need a plan. In jobs, use the `failed()` method. In HTTP calls, check the response after the retry chain completes. Don't let silent failures accumulate. And if the external service returns a `Retry-After` header or has documented rate limits, coordinate your retry delay accordingly. Blindly retrying against a rate-limited API just burns through your quota faster.

Finally, track when retries happen and how often they succeed. Excessive retries often signal deeper problems: an unstable dependency, a misconfigured integration, or infrastructure that needs attention.

## The Pattern Across Laravel

Retry appears in at least four distinct places in the framework: database transactions, the HTTP client, the queue system, and the general-purpose helper. Each implementation is tailored to its context.

- Transactions catch deadlocks.
- HTTP retries support delays and filtering.
- Jobs offer time-based windows.

But the underlying idea is identical. You learn the concept once, and you recognize it everywhere else.

## Chapter 3: Failover Mechanism

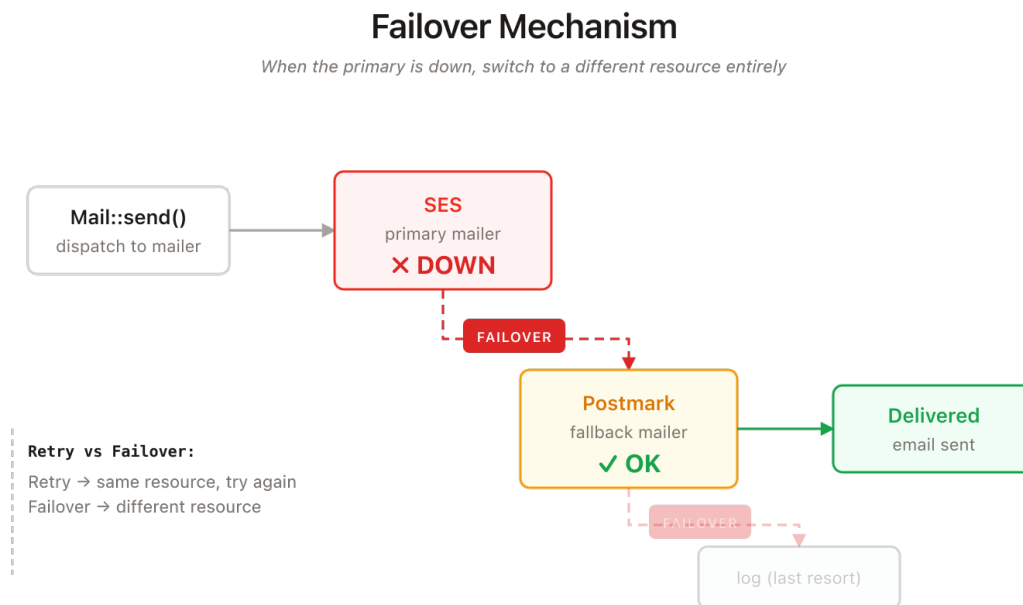
Your application sends emails through Postmark. One morning, Postmark has an outage. Every email your app tries to send fails. Users don't get password resets, order confirmations pile up, and your support queue starts filling.

Since Postmark uptime is out of your hands, the practical fix is 'failover': a backup mailer that can take over when the primary one (Postmark) goes down.

Unlike retry logic, which reattempts the same action on the same resource, **failover switches to a different resource** altogether.

If Redis is down, push jobs to the database queue. If your SMTP server is unreachable, send through a different provider. Your application keeps working while the primary resource recovers.

Laravel implements failover in three subsystems: Queues, Cache, and Mail.



## Failover in Queues

If your queue connection fails (say Redis goes offline), you don't want job dispatching to break entirely. Laravel's **failover queue driver** lets you define a chain of fallback connections.

### Configuration

```
// config/queue.php
'connections' => [

    'redis' => [
        'driver' => 'redis',
        // ...
    ],

    'database' => [
        'driver' => 'database',
        // ...
    ],

    'failover' => [
        'driver' => 'failover',
        'connections' => [
            'redis',
            'database',
            'sync',
        ],
    ],

],
```

Then set in `.env`:

```
QUEUE_CONNECTION=failover
```

## How It Works

Laravel tries the first connection ([redis](#)). If pushing a job fails (e.g., Redis unavailable), it automatically tries the next one ([database](#)). If that also fails, it falls back again ([sync](#)).

## Failover Events

Laravel fires a `QueueFailedOver` event when this happens:

```
Event::listen(QueueFailedOver::class, function ($event) {
    Log::warning('Queue failed over', [
        'from' => $event->connectionName,
        'job'  => $event->job,
    ]);
});
```

**Watch out for:** If you use Horizon, it only works with Redis. You'll need to run a separate `queue:work database` worker for the fallback connection. Fallback connections need active workers. Without them, jobs get pushed successfully but sit in the table and never get processed.

Ref: <https://laravel.com/docs/master/queues#queue-failover>

## Failover in Cache

Losing your cache layer won't crash your app, but it can slow everything to a crawl. Laravel's **failover cache store** lets you define backup stores that take over when the primary becomes unreachable.

### Configuration

```
// config/cache.php
'stores' => [

    'redis' => [
        'driver' => 'redis',
        // ...
    ],

    'database' => [
        'driver' => 'database',
        'table' => 'cache',
        // ...
    ],

    'failover' => [
        'driver' => 'failover',
        'stores' => [
            'redis',
            'database',
        ],
    ],

],
```

Then set in `.env`:

```
CACHE_STORE=failover
```

## How It Works

Laravel attempts the first store ([redis](#)). If Redis throws a connection exception, it switches to [database](#). Cache operations continue without interrupting your application.

## Failover Events

```
Event::listen(CacheFailedOver::class, function ($event) {
    Log::error('Cache store failed over', [
        'from'      => $event->storeName,
        'operation' => $event->operation,
        'key'       => $event->key,
    ]);
});
```

## Cache Failover: Practical Considerations

Cache failover looks straightforward in the config, but production behavior is more nuanced than it first appears.

The first thing to understand is that **failover is not replication**. During failover, writes go to the fallback store only. When the primary comes back online, it has none of the values that were written elsewhere. Your application ends up with divergent cache states across stores. For most caching this is acceptable, but only if you treat cached data as disposable.

Recovery creates its own version of this problem. When Redis becomes available again, Laravel switches back immediately, but the fallback store may still contain data that Redis doesn't. This can cause stale reads, sudden cache misses, and stampedes when keys get repopulated. Using [remember\(\)](#) or [rememberForever\(\)](#) helps smooth out the transition.

It's also worth noting that failover only triggers on hard failures. If Redis is online but slow, overloaded, or responding with partial errors, Laravel won't switch. And in a multi-server environment, some servers may still be talking to Redis while others have already failed over. There's no coordination between them, so you can get temporarily inconsistent reads across your cluster.

The most dangerous pitfall, though, is assuming your fallback store can do everything Redis can. Redis supports atomic increments, precise TTLs, Lua scripting, and locking primitives. File or database cache does not. If your application relies on any of these (atomic counters, cache-based locks, rate limiting, job throttling), then failing over silently changes behavior in ways that can cause subtle bugs or data corruption. Atomic locks, distributed locks, and uniqueness guards should never run through a failover cache store.

Cache failover is best treated as a last resort. **Your fallback store should be something you hope to never actually use, just enough to keep the application alive during a Redis outage.** If you find yourself regularly depending on the fallback, the underlying cache infrastructure needs attention.

Ref: <https://laravel.com/docs/master/cache#cache-failover>

## Failover in Mail

Email providers go down. SMTP servers become unreachable. API rate limits get hit. Laravel supports **failover mailers** so outgoing emails still get delivered when your primary provider has problems.

### Configuration

```
// config/mail.php
'mailers' => [

    'smtp' => [
        'transport' => 'smtp',
        'host' => env('MAIL_HOST', 'smtp.example.com'),
        // ...
    ],

    'postmark' => [
        'transport' => 'postmark',
        // ...
    ],

    'failover' => [
        'transport' => 'failover',
        'mailers' => [
            'smtp',
            'postmark',
        ],
    ],

],
```

And in your `.env`:

```
MAIL_MAILER=failover
```

## How It Works

Laravel tries `smtp` first. If sending fails (connection issues, authentication failures, timeout), it tries `postmark`. If all mailers fail, it throws the final exception.

You can stack as many backup mailers as you want. A common production setup:

```
'failover' => [  
    'transport' => 'failover',  
    'mailers' => [  
        'ses',  
        'postmark',  
        'log',  
    ],  
],
```

SES as primary, Postmark as backup, and `log` as the last fallback so no email is silently lost. The `log` mailer obviously doesn't deliver anything to users, but it gives you a record you can use to resend later.

Ref: <https://laravel.com/docs/master/mail#failover-configuration>

## Why Failover Is a Cross-Cutting Pattern

Failover in Queue, Cache and Mail isn't three separate features that happen to look similar. It's a single concept that Laravel applies across these subsystems:

| Subsystem | What Failover Protects Against                           |
|-----------|----------------------------------------------------------|
| Queue     | Job dispatching continues if Redis or the DB queue fails |
| Cache     | Caching keeps working during store outages               |
| Mail      | Email delivery survives a provider outage                |

The structure is the same every time: an ordered fallback list, automatic switching on failure, events for observability, and graceful degradation instead of a hard crash.

### Best Practices

**Monitor failover events.** Always log or alert when failovers occur. They're a signal that something in your infrastructure is unstable, even if your app is still running fine.

Choose fallbacks carefully. A fallback that also fails defeats the purpose. And don't chain too many layers; two or three is usually plenty. A long chain adds complexity and increases the time before you get either a success or a definitive failure.

**Test failover in staging.** Simulate real failures: turn off Redis, block the mail provider's API, drop cache connections. See what actually happens. Failover logic that's never been tested is failover logic you can't trust.

Keep in mind the performance tradeoffs. Database caching is slower than Redis. The `sync` queue driver means jobs run inline during the request. The `log` mailer doesn't actually send emails. Your fallback keeps the system alive, but it's running in a degraded state, not a replacement for fixing the primary.

### How Failover Differs from Retry

These two patterns complement each other, but they solve different problems:

| <b>Pattern</b> | <b>Strategy</b>             | <b>Use Case</b>                                  |
|----------------|-----------------------------|--------------------------------------------------|
| Retry          | Try the same resource again | Transient issues (network glitch, brief timeout) |
| Failover       | Switch to another resource  | When the primary system is down                  |

In practice, you often want both. Retry handles the momentary glitch; failover handles the outage.

## Chapter 4: Atomic Locks & Concurrency Control

You deploy a scheduled command that generates a daily report. It works perfectly until you scale to three servers. Now the command runs three times simultaneously, and your users get three copies of the same email.

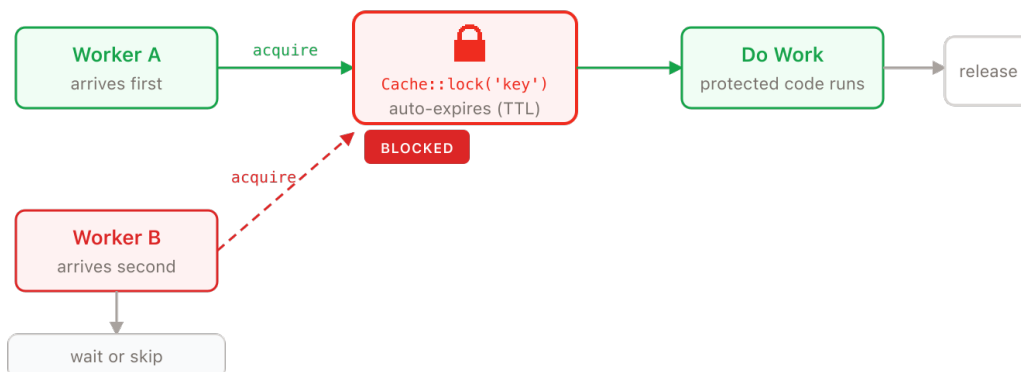
Or you have a queue job that processes a refund. Two workers pick up the same related job at the same time, and the customer gets refunded twice.

These are concurrency bugs. They only show up when multiple processes try to do the same thing simultaneously, and they almost never appear in development. Laravel's answer is **atomic locks**: a way to say "only one process can do this right now."

What makes locks interesting as a cross-cutting concept is how widely Laravel reuses them. The same locking mechanism that powers `Cache::lock()` also prevents overlapping queue jobs, stops scheduled tasks from running twice, isolates Artisan commands, and serializes session writes.

### Atomic Locks & Concurrency Control

*Only one process can hold the lock at a time — others must wait or give up*



## What an Atomic Lock Is

An atomic lock is a 'mutually exclusive claim' on a named resource:

- Only one process can hold the lock at a time.
- Other processes must wait or give up.
- Locks automatically expire after a timeout, so a crashed process can't block everyone forever.

Laravel's lock implementation is built on its cache layer. If your cache store supports atomic operations (Redis, Memcached, DynamoDB via packages), you get real distributed locks that work across multiple servers and processes.

The core API looks like this:

```
$lock = Cache::lock('import-users', 10); // TTL = 10 seconds

if ($lock->get()) {
    try {
        processUsers();
    } finally {
        $lock->release();
    }
}
```

Because the abstraction lives in the cache layer, Laravel can reuse it across many subsystems without duplicating code.

Not all cache stores support true atomic locks. For anything where concurrency actually matters, you need Redis.

Ref: <https://laravel.com/docs/master/cache#atomic-locks>

## Where Laravel Uses Locks

Once you know what a lock is, you start seeing the same mechanism in many places across the framework. Each of the following features solves a different problem, but they all use the same underlying lock abstraction.

## Preventing Overlapping Queue Jobs

Queue jobs may run concurrently across multiple workers. If two workers pick up jobs for the same user at the same time, you can end up with duplicate emails, double-billing, or a corrupted state.

Laravel provides `WithoutOverlapping` middleware to prevent this:

```
use Illuminate\Queue\Middleware\WithoutOverlapping;

public function middleware()
{
    return [
        (new WithoutOverlapping($this->userId))->expireAfter(60)
    ];
}
```

Only one job with this key runs at a time (`$this->userId` creates the unique key for this job in the above example). Under the hood, it's a lock stored in Redis, the same `Cache::lock()` mechanism wrapped in job middleware.

Ref: <https://laravel.com/docs/master/queues#preventing-job-overlaps>

## Unique Jobs: Locking at Dispatch Time

`WithoutOverlapping` prevents concurrent execution. **Unique jobs** go further: they prevent duplicate jobs from even entering the queue.

```
class ProcessUser implements ShouldQueue, ShouldBeUnique
{
    public $uniqueFor = 3600; // (optional) TTL
}
```

Laravel acquires a lock **before** dispatching. If the lock already exists, the job is silently dropped. If the lock is available, the job is queued and held.

This prevents queue floods when the same event fires repeatedly: a webhook that retries, a button a user clicks three times, a cron job that dispatches faster than workers can process.

Ref: <https://laravel.com/docs/master/queues#unique-jobs>

## Task Scheduling

Scheduled tasks often run on every server in your cluster.

```
$schedule->command('summary:generate')
    ->everyMinute()
    ->withoutOverlapping();
```

Without coordination, `summary:generate` would run three times on three servers. `withoutOverlapping` can prevent it.

Laravel acquires a lock before executing the command. If the previous run hasn't finished, the new one is skipped. Same mechanism, different context.

Ref: <https://laravel.com/docs/master/scheduling#preventing-task-overlaps>

## Isolatable Artisan Commands

Artisan commands can be marked as isolated, ensuring only one instance runs at a time:

```
protected bool $isolated = true;
```

Or via the command signature:

```
protected $signature = 'users:import {--isolated}';
```

Laravel creates a lock using the command name and arguments. This prevents multiple parallel imports from stomping on each other, overlapping cleanup tasks, or generating duplicate reports. Useful both locally (running the same command in two terminals) and in distributed environments.

Ref: <https://laravel.com/docs/master/artisan#isolatable-commands>

## Isolated Migrations

When deploying to multiple servers, two servers might attempt to run migrations simultaneously. Laravel prevents this with a lock:

```
php artisan migrate --isolated
```

One server runs the migrations. The others wait or skip. Schema state stays consistent.

Ref: <https://laravel.com/docs/master/migrations#running-migrations>

## Session Blocking

In stateful applications, multiple concurrent requests from the same user can conflict when writing to the session. Picture a user with three browser tabs, each submitting a form simultaneously. Each request reads the session, modifies it, and writes it back. The last write wins, and the other two updates are silently lost.

Laravel supports **session blocking**, which uses locks to serialize requests per session:

```
Route::post('/order', function () {  
    // ...  
})->block($lockSeconds = 10, $waitSeconds = 10);
```

One session, one lock, one request at a time. Cart update bugs and multi-tab conflicts disappear.

Ref: <https://laravel.com/docs/master/session#session-blocking>

## Considerations When Using Locks

Locks introduce coordination overhead and should be used intentionally.

**Choose lock TTLs carefully.** A lock's TTL determines how long it stays held if the process crashes without releasing it. Too short, and two processes may overlap. Too long, and a crashed process blocks progress for minutes. Aim for slightly longer than the worst-case execution time of the protected code.

Not every problem needs mutual exclusion. Locks add latency and reduce throughput under high concurrency. If the operation is idempotent (meaning running it twice produces the same result), you might not need a lock at all. An idempotent job that occasionally runs twice is often simpler and more scalable than one protected by distributed locking.

**Distributed locks aren't perfect.** Network delays, worker crashes, and backend outages can all cause surprising behavior. A lock holder might crash without releasing. A process might wait longer than expected with `block()`. A TTL might expire before the work finishes. These are realities of distributed systems, not bugs in Laravel. Design your code so the protected operation stays safe even if the lock behaves unexpectedly. Validate business invariants, and don't assume the lock alone guarantees correctness.

## **The Pattern Across Laravel**

Atomic locking appears in cache, queues, unique jobs, scheduling, Artisan commands, migrations, sessions, and Horizon, all of which use the same underlying mechanism.

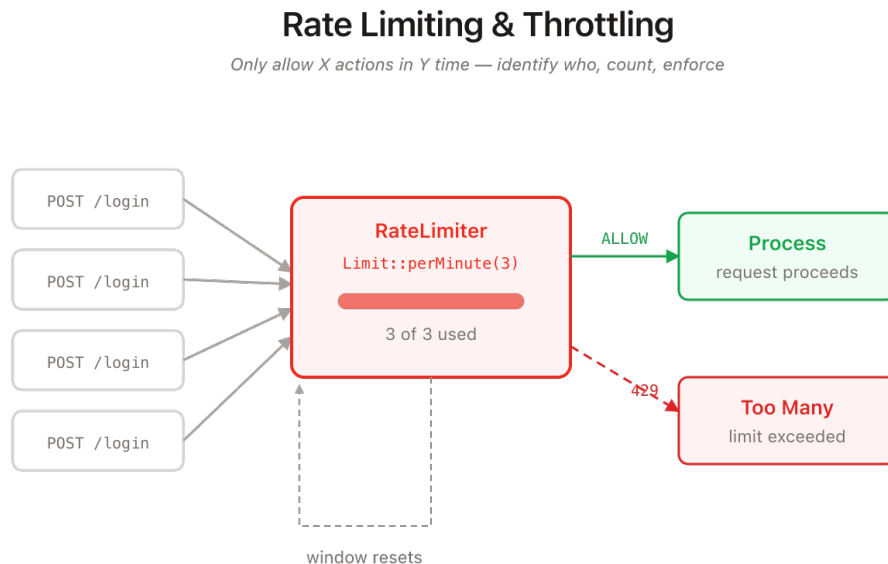
Every one of these features solves a different problem. But they all share the same shape: acquire a named lock, do the work, release the lock.

Laravel solved concurrency coordination once and then applied the same solution wherever it was needed.

## Chapter 5: Rate Limiting & Throttling

Rate limiting seems simple on the surface: only allow X actions in Y time. But it's useful in more places than you'd expect once you see how Laravel applies it. It protects your app from abuse, prevents unnecessary load, and keeps noisy users or background jobs from overwhelming your system.

Laravel gives you **one unified abstraction** for all of this. HTTP requests, exception reporting, queue jobs, and your own internal logic all build on top of it. Learn the pattern once, and you can apply it everywhere.



### Why Rate Limiting Exists

Most systems have limits. Databases, APIs, authentication endpoints, and notification providers can only handle so many operations at once before performance degrades or costs start climbing.

Rate limiting lets you shape traffic so your system stays healthy even when usage spikes or a single user behaves aggressively. It smooths out unpredictable workloads and prevents short bursts of activity from turning into outages.

Some common scenarios where rate limiting helps:

Login endpoints, third-party APIs, and database-heavy routes often need strict rate limits so they don't get overwhelmed. Per-user limits prevent one aggressive client from consuming 90% of your API capacity. Simple ceilings catch accidental floods from misconfigured cron jobs or buggy mobile apps. And even basic limits like "5 login attempts per minute" stop brute-force attempts before they become a real problem.

The logic is always the same: identify *who* is trying to do something, decide *how often* they can do it, count the attempts, and enforce the limit when it's exceeded. Instead of letting your system collapse under load, rate limiting slows things down in a controlled, predictable way.

## The RateLimiter Abstraction

The central API looks like this:

```
RateLimiter::for('login', function (Request $request) {  
    return Limit::perMinute(5)->by($request->ip());  
});
```

This captures the entire mental model:

- **What** you're limiting (`login`)
- **How often** (5 per minute)
- **Who** the rule applies to (`by($request->ip())`)

Laravel handles the rest: tracking counts, storing them (usually in Redis), and deciding whether to allow or reject the action.

This same abstraction powers HTTP throttling, job throttling, notification throttling, and any custom limits you define yourself.

Ref: <https://laravel.com/docs/master/rate-limiting>

## HTTP Request Throttling

The most familiar example is route throttling:

```
Route::middleware(['throttle:60,1'])->group(function () {  
    // ...  
});
```

Or, with custom-named limiters:

```
Route::post('/login')->middleware('throttle:login');
```

Under the hood, everything flows through the same `RateLimiter` engine, so your entire app shares a consistent rate-limiting mechanism across features.

Ref: <https://laravel.com/docs/master/routing#attaching-rate-limiters-to-routes>

## Exception Throttling

Laravel supports throttling **reported exceptions**. This prevents your logs or error-tracking services from being overwhelmed when the same error repeats rapidly, for example, when an external API goes down and every request to it fails.

Exception throttling is configured in `bootstrap/app.php`:

```
$app->withExceptions(function ($exceptions) {  
    $exceptions->throttle(function (Throwable $e) {  
        return Lottery::odds(1, 1000); // sample 1 in 1000  
    });  
});
```

You can return either a **Lottery** (random sampling) or a **Limit** (fixed rate limiting). This makes it easy to cut down noise from high-volume exceptions without fully hiding the problem.

A common pattern is to throttle only specific, noisy exception types:

```
$exceptions->throttle(function (Throwable $e) {  
    if ($e instanceof ExternalApiException) {  
        return Limit::perMinute(200);  
    }  
});
```

Critical errors still get reported, while repetitive ones are kept under control. Exception throttling isn't a replacement for monitoring. It keeps your reporting systems healthy during spikes in failures.

Ref: <https://laravel.com/docs/master/errors#throttling-reported-exceptions>

## Throttling Queue Jobs

Laravel gives you two ways to throttle queue jobs, each solving a different problem.

### Rate Limiting Jobs

Rate limiting controls **how many jobs** can run over a certain period. This is useful when jobs call an API with strict limits or when you want to smooth out bursts of work.

```
use Illuminate\Queue\Middleware\RateLimited;  
  
public function middleware()  
{  
    return [ new RateLimited('external-api') ];  
}
```

You define the limiter using `RateLimiter::for(...)`, and Laravel automatically releases jobs back to the queue when they hit the limit. Throughput stays predictable without you manually managing delays.

Ref: <https://laravel.com/docs/master/queues#rate-limiting>

### Exception-Based Throttling

Exception throttling takes a different approach. Instead of capping throughput upfront, it slows down jobs that **fail repeatedly**. If a job keeps hitting a failing API, Laravel applies backoff logic after a certain number of consecutive exceptions.

```
use Illuminate\Queue\Middleware\ThrottlesExceptions;

public function middleware()
{
    return [ new ThrottlesExceptions(10, 300) ];
}
```

If the job throws more than 10 exceptions, Laravel throttles it for 300 seconds before retrying. This gives unstable dependencies time to recover instead of hammering them with failing requests.

Ref: <https://laravel.com/docs/master/queues#throttling-exceptions>

### When to Use Which

Use **rate limiting** when the resource has known usage limits or can only handle a certain amount of throughput. Use **exception throttling** when the resource is unstable or intermittently failing. You can use both together when you need to protect external systems from overload *and* failure loops. The two middleware components compose well.

### Why Redis Matters for Throttling

Most of Laravel's throttling features rely on Redis because rate limiting needs fast, atomic operations. Every limit check (incrementing a counter, checking a timestamp,

acquiring a lock) must occur as a single atomic operation. Redis does this in memory, not on disk.

Redis provides atomic increments ([INCR](#), [INCRBY](#)) for tracking counts without race conditions, key expiry for automatically resetting limits when the window ends, and consistent behavior across multiple application servers in a cluster. Traditional databases or file-based stores can't reliably do this at scale.

## Choosing the Right Scope

Rate limiting only works well if you scope it correctly. Laravel makes it easy to define limits per user, per route, or globally, but choosing the wrong scope can block the wrong people or fail to protect anything at all.

### *Per-User Limits*

Use per-user limits when you want fairness. Each user gets their own quota, so one aggressive client can't slow down everyone else.

```
RateLimiter::for('api', function ($request) {  
    return Limit::perMinute(60)->by($request->user()->id ?: $request->ip());  
});
```

This works well for public APIs, login attempts, and any action tied to an authenticated user.

### *Global Limits*

A global limit is shared across *all* users. It's useful when you're protecting something with a system-wide capacity constraint.

```
RateLimiter::for('external-api', function () {  
    return Limit::perMinute(500);  
});
```

Use global limits for external APIs with system-wide quotas or expensive background tasks where total throughput matters more than per-user fairness.

## Hybrid Limits

Many real-world workloads need both. You can layer a per-user limit on top of a global one:

```
return [  
    Limit::perMinute(60)->by($request->user()->id),  
    Limit::perMinute(500),  
];
```

This ensures users can't overload the system, either individually or collectively.

## Using Rate Limits Beyond HTTP

Laravel's rate limiter isn't just for API requests. Because `RateLimiter::attempt()` works in plain PHP, you can use it anywhere: inside services, domain logic, console commands, or scheduled tasks.

```
use Illuminate\Support\Facades\RateLimiter;  
  
RateLimiter::attempt(  
    'import-check',  
    $perMinute = 5,  
    function () {  
        // Only runs if we're under the limit  
        dispatch(new CheckExternalImport);  
    }  
);
```

Laravel checks whether the key `import-check` has been used fewer than 5 times in the last minute. If it's within the limit, the callback runs. If the limit has been reached, the callback is skipped and the system avoids performing unnecessary work.

This pattern works anywhere you want to cap how often something fires: background tasks that shouldn't run too frequently, internal loops that hit external APIs, scripts that could accidentally spam an integration.

Rate limiting is not tied to HTTP. It's a general-purpose tool.

Ref: <https://laravel.com/docs/master/rate-limiting>

## Things to Keep in Mind

**Pick the right identity.** Do you want this limit applied per IP, per user, or globally? A wrong scope can block legitimate users or let attackers slip through. Think about this before writing the limiter.

**Tune the window.** A 1-minute window behaves very differently from a 10-second window. Smaller windows give smoother behavior; larger windows allow spikes. Match the window to the resource you're protecting.

**Rate limiting is a control mechanism, not a security guarantee.** It increases the cost of abuse, but a determined attacker can circumvent simple rate limits. Treat it as one layer in a broader defense. And expect minor drift: distributed counters aren't perfect down to the millisecond, but they're close enough for real-world needs.

Also, think about what "blocked" means in your context. Should your app hard-reject the action with a 429? Delay it? Drop it silently? The right answer depends on who's calling and what they expect.

## The Pattern Across Laravel

Rate limiting shows up in HTTP middleware, exception reporting, queue job middleware, and as a standalone tool for arbitrary code. Each use case has its own configuration, but they all flow through the same `RateLimiter` abstraction: define a limiter with `RateLimiter::for(...)`, and apply it to routes, jobs, or your own code. The mental model stays the same regardless of context.

## Chapter 6: The Pipeline Pattern

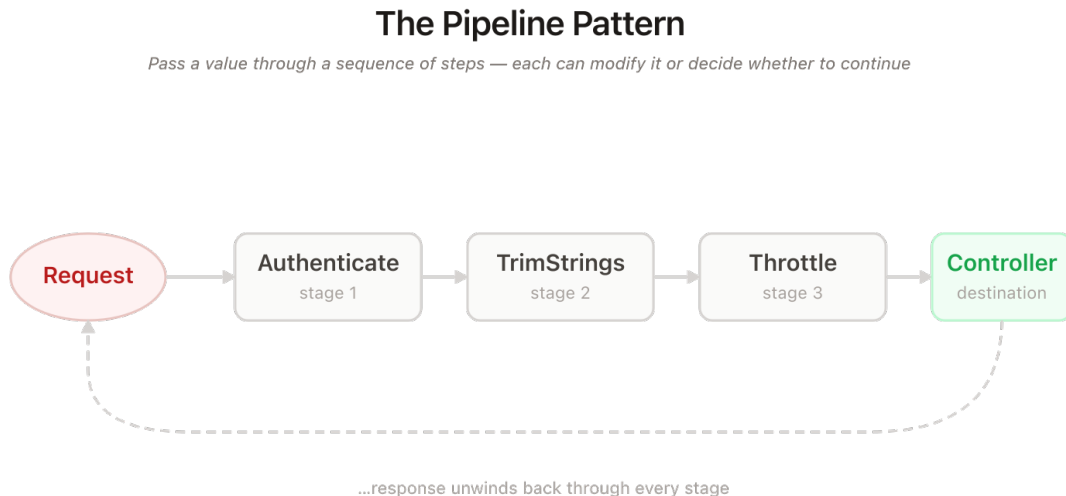
You've already used the pipeline pattern in Laravel, even if you've never called it that.

Every HTTP request that enters your application passes through a stack of middleware: authentication, CSRF protection, rate limiting, and string trimming. Each one does its job and passes the request forward. By the time a request reaches your controller, it's been shaped by a sequence of independent steps.

That's a **pipeline**.

The same idea shows up in several other places: job middleware, outgoing HTTP client middleware, process piping, and as a general-purpose class you can use in your own code.

Once you see the shape, you'll recognize it throughout the framework.



## How Pipelines Work

A pipeline is straightforward:

1. You start with an input (a request, a DTO, a model, any object).
2. You pass it through a sequence of steps.
3. Each step can modify the input or decide whether to pass it forward.
4. The final result drops out at the other end.

Each step is independent. You can add, remove, reorder, or reuse steps without touching the others.

Here's what happens internally when Laravel runs a pipeline:

1. You provide an initial value and a list of "pipes" (classes with a `handle($passable, Closure $next)` method).
2. Laravel uses `array_reduce` to wrap each pipe around the next, building a nested chain of closures.
3. Execution starts from the outermost pipe, which calls the next, which calls the next, until the final destination callback runs.
4. Whatever value the last pipe returns becomes the output of the entire pipeline.

## A Simple Example

Here's a minimal pipeline:

```
use Illuminate\Pipeline\Pipeline;

$result = app(Pipeline::class)
    ->send($input)
    ->through([
        CleanInput::class,
```

```

        ValidateData::class,
        TransformPayload::class,
    ])
    ->thenReturn();

```

Each pipe receives two arguments:

```

public function handle($payload, Closure $next)
{
    // do something with $payload
    return $next($payload);
}

```

Every piece of work is isolated, testable, and reusable. This is the same structure that appears throughout the framework in different forms.

## HTTP Middleware

When a request enters your application, it passes through a stack of middleware. Each middleware follows the same structure:

```

public function handle($request, Closure $next)
{
    // Work before passing the request forward
    return $next($request);
}

```

The HTTP kernel resolves the middleware list and sends the request through each layer in order. The response then travels back through the same stack in reverse, since each middleware layer wraps the one after it.

By the time a request reaches your controller, it's been filtered, validated, and prepared by a pipeline of reusable classes. This is the pipeline pattern at its most visible.

Ref: <https://laravel.com/docs/master/middleware>

## Job Middleware

Queue jobs support middleware too, and the structure mirrors HTTP middleware almost exactly:

```
public function middleware()
{
    return [
        new RateLimited('api'),
        new WithoutOverlapping($this->modelId),
    ];
}
```

Each middleware class receives the job and can prevent it from running, throttle it, apply locks, add retry behavior, or wrap the execution in logging.

Laravel processes job middleware using the same pipeline engine as HTTP requests, which lets you express cross-cutting queue concerns without burying them inside the job's `handle()` method.

Ref: <https://laravel.com/docs/master/queues#job-middleware>

## HTTP Client Middleware

Laravel's HTTP client uses a pipeline for **outgoing** requests. Since Laravel's HTTP client is powered by Guzzle, this is done through Guzzle middleware.

You can register middleware to manipulate the outgoing request:

```
use Illuminate\Support\Facades\Http;
use Psr\Http\Message\RequestInterface;

$response = Http::withRequestMiddleware(
    function (RequestInterface $request) {
        return $request->withHeader('X-Example', 'Value');
    }
)->get('http://example.com');
```

Or inspect the incoming response:

```
use Psr\Http\Message\ResponseInterface;

$response = Http::withResponseMiddleware(
    function (ResponseInterface $response) {
        $header = $response->getHeader('X-Example');

        // ...

        return $response;
    }
)->get('http://example.com');
```

Each middleware can log request/response pairs, modify headers, time or trace requests, inject failover logic, or wrap calls in circuit breakers. The structure mirrors server-side middleware but is applied to external service calls rather than incoming requests.

Ref: <https://laravel.com/docs/master/http-client#guzzle-middleware>

## Process Pipelines

Laravel also supports process pipelines, where you run multiple external commands in sequence, with the output of one piped into the next. This works just like 'shell piping'.

```
use Illuminate\Support\Facades\Process;
use Illuminate\Process\Pipe;

$result = Process::pipe(function (Pipe $pipe) {
    $pipe->command('cat file.txt');
    $pipe->command('grep "Laravel"');
});
```

`Process::pipe(...)` lets you define a series of commands. The stdout of the first is passed as input to the next. You can capture real-time output by passing a callback as the second argument, and you can name each step with `as(...)` for debugging.

This is useful when you need to chain shell commands ([grep](#), [awk](#), [sed](#)), run simple ETL tasks using external tools, or when you want the composability of a shell pipeline managed from within Laravel code. It's the same pipeline concept, applied at the OS level instead of PHP data.

Ref: <https://laravel.com/docs/master/processes#process-pipelines>

## The Pipeline Class

Sometimes you need a pipeline outside of HTTP or queues, for example, when processing imports, transforming data, or applying a series of business rules.

Laravel's [Pipeline](#) class handles this:

```
$result = Pipeline::send($order)
    ->through([
        ApplyDiscounts::class,
        ValidateInventory::class,
        RecalculateTotals::class,
    ])
    ->thenReturn();
```

This is useful when you want to break complex logic into independent steps without building a monolithic service class.

Ref: <https://laravel.com/docs/master/helpers#pipeline>

## When to Use Pipelines in Your Own Code

A pipeline is a good fit when:

- Logic needs to be applied in a defined order
- Each step is isolated and testable
- You expect to add or remove steps later

- You want to reuse steps across multiple workflows

Some examples from real projects: importing a CSV (sanitizing rows, validating, mapping columns, storing), running business rules on a domain object, chaining transformations on an uploaded file, or post-processing user-submitted content.

Pipelines help you push work into small, purpose-built classes instead of growing a single service class that does everything. When each step has a single job and a clean interface, the workflow becomes easier to test, modify, and read for other developers.

## **The Pattern Across Laravel**

The pipeline pattern appears in HTTP middleware, job middleware, HTTP client middleware, process piping, and the general-purpose [Pipeline](#) class. Each one passes a value through a sequence of steps, with each step deciding what to do next.

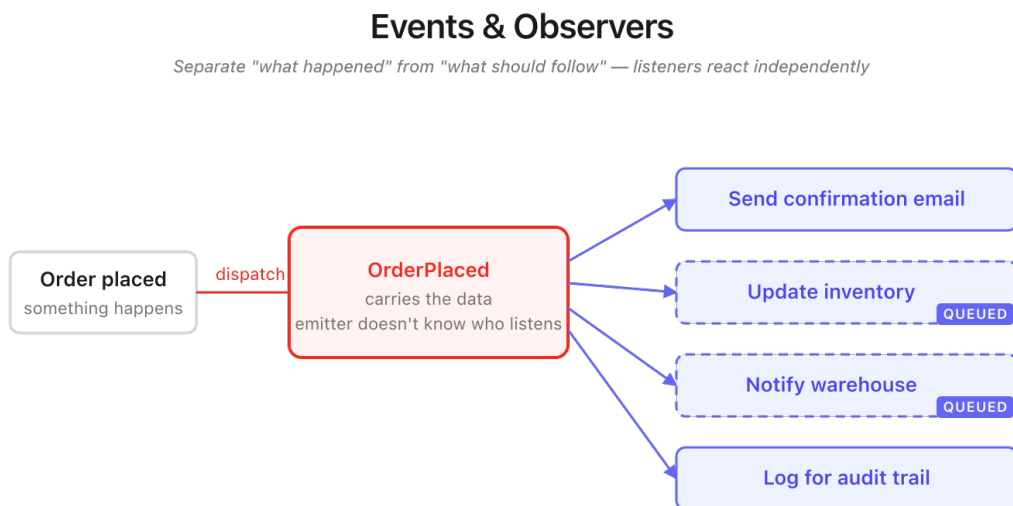
This consistency is what gives Laravel its feel of coherence across very different subsystems. Incoming requests, outgoing HTTP calls, background jobs, and shell commands all use the same structural idea. When you learn how middleware works for HTTP, you already understand how it works for jobs and outgoing requests.

## Chapter 7: Events & Observers

A user places an order. Now several things need to happen: send a confirmation email, update inventory, notify the warehouse, log the transaction for auditing. You could put all of that in the controller, but it quickly turns into a mess. Every new requirement means touching checkout logic, and the method grows until nobody wants to go near it.

Events solve this by separating "what happened" from "what should follow." You dispatch an event that says "an order was placed," and any number of listeners can react to it independently. The checkout code doesn't know or care what happens next.

Laravel uses this pattern everywhere. Model lifecycle hooks, queue job tracking, authentication logging, broadcasting, notifications, scheduled task monitoring, and outbound HTTP observation. They're all built on the same idea: something happens, an event fires, and anything that cares can respond.



## What Events Solve

Without events, side effects tend to accumulate inside controllers, model methods, or service classes. Sending an email after registration is part of the registration method. Syncing a search index after a model update lives in the update method. Logging an audit trail lives wherever the action happens.

This creates tight coupling. Adding new behavior means modifying existing code. Testing one thing means dealing with all its side effects. And large service classes are hard to reason about because they do too much.

Events give you:

- **Loose coupling** – the emitter doesn't know who is listening
- **Clear separation** – what happened vs. what should happen next
- **Easy extensibility** – add or remove listeners without touching the original code
- **Optional async** – turn any listener into a queued job by implementing `ShouldQueue`

## How Events Work

The flow is minimal:

1. An event is dispatched, usually an object describing what happened.
2. The dispatcher finds listeners registered for that event.
3. Listeners execute, either synchronously or asynchronously.
4. Queued listeners are handed off to the queue worker with retry and backoff support.

The event carries the data. The listeners carry the reactions.

## Using Events in Your Own Code

### 1. Create an event

```
class OrderPlaced
{
    public function __construct(
        public Order $order
    ) {}
}
```

### 2. Create listeners

```
class SendOrderConfirmation implements ShouldQueue
{
    public function handle(OrderPlaced $event)
    {
        Mail::to($event->order->user)->send(new OrderReceipt($event->order));
    }
}

class UpdateInventoryLevels implements ShouldQueue
{
    public function handle(OrderPlaced $event)
    {
        $event->order->items->each->reduceInventory();
    }
}
```

### 3. Register listeners

```
// EventServiceProvider
protected $listen = [
    OrderPlaced::class => [
        SendOrderConfirmation::class,
        UpdateInventoryLevels::class,
    ],
]
```

```
];
```

#### 4. Dispatch the event

```
OrderPlaced::dispatch($order);
```

You can add new listeners later without touching the checkout logic. Heavy work like emailing and stock updates happens asynchronously. Everything stays testable because each listener does one thing.

#### How Laravel Itself Uses This Pattern

One reason Laravel feels consistent is that it uses the same event pattern internally across many subsystems.

#### *Model Observers*

Observers apply the event pattern to Eloquent model lifecycle hooks: `creating`, `created`, `updating`, `updated`, `deleting`, `deleted`, `restoring`, and others.

Instead of placing side effects inside your models, observers keep them centralized and separate. A typical observer might generate slugs, sync search indexes, clear related cache, log changes, or dispatch domain events.

Observers keep your models lean. The model defines the data and relationships; the observer handles the reactions.

#### Queue Events

The queue worker dispatches events during every job lifecycle stage: `JobProcessing`, `JobProcessed`, `JobFailed`, `JobExceptionOccurred`. Tools like Horizon listen to these to track metrics and render dashboards. You can hook into the same events for your own monitoring or alerting.

## **Authentication Events**

Events like [Registered](#), [Login](#), and [Logout](#) let you hook into user lifecycle changes without modifying the authentication system. Log activity, detect suspicious behavior, trigger welcome emails, build analytics pipelines.

## **Broadcasting**

Broadcastable events are normal events with additional metadata, allowing Laravel to send them via websockets or Pusher. The event system does double duty: local listeners handle server-side reactions, while the broadcasting layer pushes the same event to the browser.

## **Notifications**

When notifications are sent, Laravel fires [NotificationSending](#) and [NotificationSent](#). These let you log delivery, track open rates, or add custom side effects without touching notification classes.

## **Task Scheduling**

The scheduler emits events before and after tasks run: [ScheduledTaskStarting](#), [ScheduledTaskFinished](#), [ScheduledTaskSkipped](#), [ScheduledTaskFailed](#). These are useful for monitoring execution, sending alerts on failure, or logging task history.

## **HTTP Client**

Laravel's HTTP client dispatches [RequestSending](#), [ResponseReceived](#), and [ConnectionFailed](#) when making outbound requests. This gives you hooks for global request logging, performance monitoring, debugging failing services, or tracking outbound API usage.

## **Context Events**

Laravel's Context system fires [ContextStarted](#), [ContextFlushed](#), and [ContextUpdated](#) as context is manipulated. These are useful when working with

background tasks or long-running processes where context changes need to be observed.

## ***The Common Thread***

Each of these subsystems solves a completely different problem. But they all reuse the same structure: something happens, Laravel emits an event, developers or packages attach listeners, and no tight coupling is required. Learning the pattern in one place means understanding it everywhere else.

## **Queued Listeners**

Listeners are generally executed immediately. But that doesn't mean they are limited to processing small tasks. Turning a listener into a background job takes only one step: adding the `ShouldQueue` interface:

```
class SendOrderConfirmation implements ShouldQueue
```

Laravel handles dispatching to the queue, serialization, retry attempts, backoff strategies, and failure handling. This is one of the easiest ways to improve request performance without restructuring your code.

## **When to Use Events, and When Not To**

Events work well when multiple things need to react to the same action, when you want extensibility without modifying existing code, when you want to move heavy tasks off the main request, or when the reactions are independent of each other (email, indexing, and audit logs don't need to coordinate).

They're less suited when core business rules depend on the result of the listener, when too many side effects make the event flow hard to trace, when you need strict execution order for reactions, or when you need immediate synchronous guarantees about what happened.

Use events to enhance workflows. Don't use them to hide essential business logic in scattered listeners where the control flow becomes invisible.

## **The Pattern Across Laravel**

Events and observers show up in models, queues, authentication, broadcasting, notifications, scheduling, the HTTP client, and the context system. Each subsystem fires its own events, but the mechanism is always the same: dispatch an event object, let registered listeners react.

This separation of "what happened" from "what follows" is one of the most reused structural ideas in the framework. It keeps individual components focused while allowing the rest of the application to respond without tight coupling.

## Chapter 8: Driver Pattern

Your application caches data with Redis in production. In your test suite, you use the array driver. In a staging environment, you might use the database. Your code doesn't change between any of these:

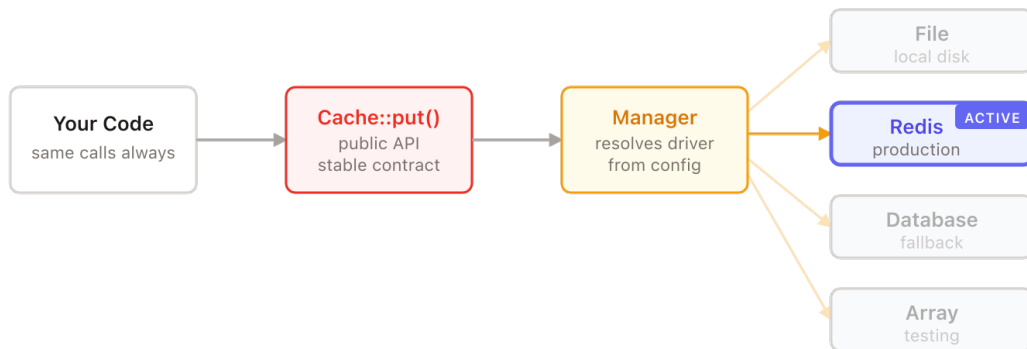
```
Cache::put('key', 'value', 3600);
```

This works because Laravel separates *what you want to do* from *how it gets done*. You write against a stable API. The backend is a configuration detail.

Laravel calls these interchangeable backends **drivers** (or sometimes stores, disks, connections, or channels, depending on the subsystem). The pattern appears throughout the framework, and it's one of the reasons switching infrastructure in a Laravel app is usually a config change rather than a rewrite.

### Driver Pattern

*One stable API, interchangeable backends — switching is a config change, not a rewrite*



## How Laravel Structures This

Most driver-based systems in Laravel follow the same four-part structure:

### 1. A Public API

This is what you interact with, usually through a facade:

```
Cache::get('key');  
Storage::put('file.txt', $data);  
Mail::send(...);
```

### 2. A Manager

The manager resolves which driver to use based on configuration:

```
Cache::store('redis');
```

When you call `Cache::store('redis')`, the `CacheManager` instantiates the Redis cache driver and returns it. Every driver-based subsystem has a corresponding manager class.

### 3. Drivers (Concrete Implementations)

Each driver implements the same contract but talks to a different backend. For example, the Redis cache driver and the file cache driver both implement `Illuminate\Contracts\Cache\Store`, but one talks to Redis and the other writes to disk.

### 4. Configuration

The active driver is selected in config:

```
'default' => env('CACHE_STORE', 'redis'),
```

That's the entire structure. Everything else is hidden behind the public API.

## Where This Pattern Appears

The driver/store pattern appears in nearly every Laravel subsystem that interacts with external infrastructure. Here's a survey of where you'll encounter it.

### Cache

```
Cache::put('key', 'value');
```

Drivers: `redis`, `memcached`, `file`, `database`, `array`

The `array` driver is particularly useful in tests – fast, disposable, no external dependencies.

### Queue

```
dispatch(new ProcessOrder);
```

Drivers: `redis`, `database`, `sqs`, `sync`, `null`

The `sync` driver runs jobs inline during the request, which helps when debugging. The `null` driver silently discards jobs, which is useful in test suites where you want to assert that a job was dispatched without actually running it.

### Filesystem

```
Storage::put('file.jpg', $data);
```

Drivers: `local`, `s3`, `ftp`, `sftp`

You can develop against the `local` driver and deploy to S3 without changing application code. This is one of the places where the driver pattern pays off most visibly.

## **Mail**

```
Mail::to($user)->send(new OrderReceipt($order));
```

Drivers: `smtp`, `ses`, `mailgun`, `postmark`, `log`

The `log` driver writes the email content to your log file instead of sending it, useful during development when you want to verify emails without actually delivering them.

## **Sessions**

```
session(['key' => 'value']);
```

Drivers: `file`, `cookie`, `database`, `redis`

## **Notifications**

```
$user->notify(new InvoicePaid);
```

Channels: `mail`, `database`, `sms`, `slack`, `broadcast`

Notification "channels" are the notification system's version of drivers. Each channel implements the same `send()` contract but delivers through a different medium.

## **Logging**

```
Log::info('Something happened');
```

Channels: `single`, `daily`, `stack`, `syslog`, `slack`

The `stack` channel lets you combine multiple log channels. You can write to a file and send to an external service simultaneously.

## Why This Pattern Matters

### ***Swap infrastructure without rewriting application code***

When your code talks to a contract rather than a specific service, the backend becomes a configuration value. You can move from local storage to S3, switch from Redis to Memcached, or replace SES with Postmark by changing one line in your config or `.env`. Application code stays the same.

### ***First-class testing support***

Because your code is written against contracts, Laravel can offer fake implementations that capture calls without hitting real services:

```
Cache::fake();  
Mail::fake();  
Queue::fake();
```

Tests run faster and don't require external infrastructure. You assert against what was called, not what happened in Redis or Mailgun.

### ***Easy extensibility***

If you need a custom backend, you extend the manager:

```
Cache::extend('custom', fn () => new CustomCacheStore());
```

Your new driver plugs into the existing system. The rest of your code still calls `Cache::get()` and `Cache::put()` the same way it always did.

### ***Clean separation of concerns***

Your code expresses intent: "cache this," "send this," "queue this." The driver handles the mechanics: how it's stored, which provider sends it, where jobs are pushed. That separation means you can change the mechanics without touching business logic.

## The Terminology Varies

Not every subsystem uses the word "driver." The concept is the same, but Laravel uses different names depending on the context:

| Subsystem              | Term Used                 |
|------------------------|---------------------------|
| Cache, Sessions        | <b>Store</b>              |
| Filesystem             | <b>Disk</b>               |
| Queue, Database        | <b>Connection</b>         |
| Notifications, Logging | <b>Channel</b>            |
| Mail                   | <b>Transport / Mailer</b> |

If you see any of these in Laravel's config files, you're looking at the driver pattern.

## Applying This Pattern in Your Own Code

The driver pattern isn't just something Laravel uses internally. It's worth reaching for any time your application needs to support multiple backends for the same operation.

Some examples:

- **Payment gateways** – Stripe, PayPal, SSLCommerz
- **SMS providers** – Twilio, Vonage, local gateway
- **Search engines** – Algolia, Meilisearch, Elasticsearch
- **AI providers** – OpenAI, Anthropic, local models
- **Export formats** – CSV, Excel, PDF

The approach follows the same four-part structure described above. Define a contract that describes what the operation does. Write drivers that implement the contract for each backend. Use a manager or factory to resolve the right driver from configuration. Your application code talks to the contract, not the driver.

Laravel provides `Illuminate\Support\Manager` as a base class you can extend for your own managers, so you don't need to build the driver-resolution logic from scratch.

## **The Pattern Across Laravel**

The driver/store pattern is one of the most visible structural decisions in the framework. Cache, queues, filesystems, mail, sessions, notifications, and logging all use it. The API you write against stays stable while the backend changes based on a config value.

This is also why Laravel can support failover (covered in an earlier chapter). When you define a failover cache store or failover mailer, you're giving Laravel an ordered list of drivers to try. The failover mechanism works because all drivers behind a given facade implement the same contract. Without the driver pattern, failover wouldn't be possible.

## Chapter 9: Deferring Work

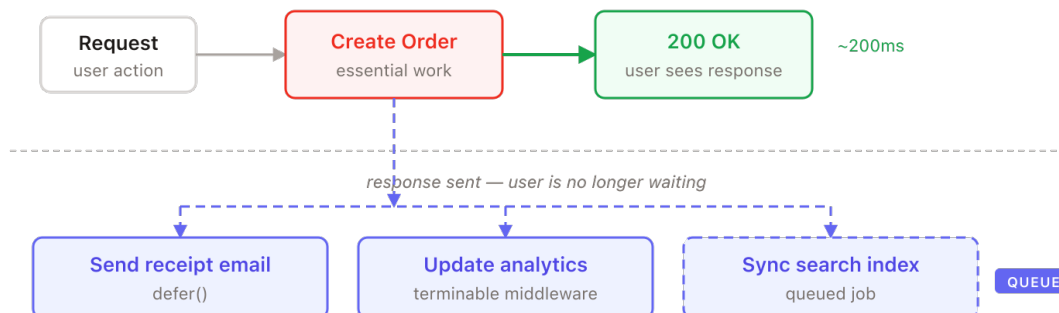
A user places an order. Your application needs to send a confirmation email, update analytics, and log the transaction. None of that needs to happen before the user sees their "Order confirmed" response. But if you do it all synchronously, before sending the response to the user, the response takes two seconds instead of two hundred milliseconds.

Deferring work means 'shifting execution away from the critical path'. Instead of doing everything before responding, you respond first and handle the rest afterward. Or you define the work up front but only execute it when it's actually needed.

You've already seen the most complete version of this idea in queues. But Laravel also provides lighter-weight ways to defer work without introducing full background processing. These are useful when you need something done "soon" but don't need the retry logic, persistence, or worker infrastructure that queues provide.

### Deferring Work

*Respond first, handle the rest afterward — keep the critical path fast*



## After-Response Execution

The most direct form of deferring happens after an HTTP response has been sent to the user. The PHP process is still alive, but the user isn't waiting anymore.

### *Terminable Middleware*

Middleware can define a `terminate()` method that runs after the response:

```
class LogRequest
{
    public function handle($request, Closure $next)
    {
        return $next($request);
    }

    public function terminate($request, $response)
    {
        logger('Request finished');
    }
}
```

`handle()` runs during the request. `terminate()` runs after the response is sent. This is useful for logging, metrics collection, or any cleanup that doesn't affect what the user sees.

Ref: <https://laravel.com/docs/master/middleware#terminable-middleware>

### *The defer() Helper*

The `defer()` helper is the simplest way to push work to after the response:

```
$order = Order::create([...]);

defer(fn () => Mail::to($order->user)->send(new Receipt($order)));

return response()->json(['ok' => true]);
```

The response goes out immediately. The email is sent afterward, still in the same PHP process.

Ref: <https://laravel.com/docs/master/helpers#deferred-functions>

## **Dispatching Jobs After Response**

You can also dispatch jobs to run after the response:

```
SendEmail::dispatchAfterResponse($user);
```

or

```
dispatch(function () {  
    // do something  
})->afterResponse();
```

This runs after the HTTP response is sent, but still within the same PHP process. It is not a real background job. There are no retries, no persistence, no queue worker involved. If the process dies, the work is lost.

Historically, Laravel supported this via `dispatchAfterResponse()`, but the newer `defer()` helper is more explicit and flexible for the same purpose. `dispatchAfterResponse()` is no longer documented, though the functionality still exists through the `Dispatchable` trait.

## **Deferring Events**

Events can also be deferred until after the response:

```
OrderPlaced::dispatchAfterResponse($order);
```

This is useful when the event triggers heavy listeners and the result isn't needed during the request. Like `dispatchAfterResponse()` for jobs, this uses the same `Dispatchable` trait and is no longer documented, but still works.

## Deferring HTTP Client Batches

When you need to make multiple outbound HTTP requests, Laravel lets you batch them and defer execution:

```
use Illuminate\Http\Client\Batch;
use Illuminate\Support\Facades\Http;

$responses = Http::batch(fn (Batch $batch) => [
    $batch->get('http://localhost/first'),
    $batch->get('http://localhost/second'),
    $batch->get('http://localhost/third'),
])->then(function (Batch $batch, array $results) {
    // All requests completed successfully...
})->defer();
```

The `defer()` call tells Laravel to execute the batch after the response has been sent to the user, not during the request.

Ref: <https://laravel.com/docs/master/http-client#deferring-batches>

## Deferred Service Providers

Laravel applies the same idea at the framework level. Some service providers are deferred, meaning they only load when something actually resolves one of their bindings:

```
class RiakServiceProvider extends ServiceProvider implements DeferrableProvider
{
    public function register(): void
    {
        $this->app->singleton(Connection::class, function (Application $app) {
            return new Connection($app['config']['riak']);
        });
    }
}
```

```

    }

    public function provides(): array
    {
        return [Connection::class];
    }
}

```

Instead of loading every service provider at boot, Laravel skips deferred providers until their services are actually requested. This reduces initial boot time and avoids loading code that may never be used during a given request.

Ref: <https://laravel.com/docs/master/providers#deferred-providers>

## Defer vs. Queue

Both defer work for later. The difference is in the guarantees they provide.

|                            | Defer                           | Queue                  |
|----------------------------|---------------------------------|------------------------|
| Runs later                 | Yes                             | Yes                    |
| Runs in a separate process | No                              | Yes                    |
| Survives process death     | No                              | Yes                    |
| Retry support              | No                              | Yes                    |
| Best for                   | Lightweight post-response tasks | Heavy or critical work |

Deferring is a good fit for sending non-critical emails, logging and analytics, cache warming, lightweight external calls, and tasks where the user doesn't need to see the result.

Avoid deferring when the task is critical and must complete, when you need retries or failure handling, when the work is heavy enough to delay the process shutdown, or when you need visibility into whether the task succeeded. In those cases, use queues.

## The Pattern Across Laravel

Deferring work shows up in terminable middleware, the `defer()` helper, after-response job dispatch, HTTP client batch deferral, and deferred service providers. Each one solves a different problem, but they all share the same idea: move non-essential work off the critical path so the main flow stays fast.

The distinction between "do it now" and "do it later" is one of the simplest and most useful decisions you can make when building an application. Laravel gives you several tools to make that decision at different levels of the framework.

## Conclusion

This book has spent nine chapters showing you the ideas Laravel reuses everywhere. By now, you're probably seeing them in places we didn't even mention.

That's the point. Pattern recognition compounds.

But we'd be doing you a disservice if we ended the book there.

Patterns are lenses. They help you see the structure that was always there. But lenses also distort. When you get good at recognizing patterns, you start reaching for them even when they don't fit.

You see a problem and think "that's a pipeline" when it's really just a loop. You add an event system when a method call would do. You build a driver abstraction for something that will have only one implementation.

We've both done all of these.

The most common mistake is **premature abstraction**. You read about the driver pattern, and suddenly your payment integration needs a full manager class, a contract, a config file, and three drivers, even though you only accept Stripe.

You'll know you need the abstraction when the second backend shows up. Not before.

The second mistake is **hiding control flow behind events**. Events are excellent for decoupling side effects from business logic. But when the *business logic itself* lives in event listeners scattered across six files, you've traded a readable method for an invisible chain of reactions.

When a new developer asks, "What happens when an order is placed?" the answer shouldn't require a treasure hunt.

The third mistake is **adding locks, retries, and rate limiters** because you can, not because the problem calls for it. Every pattern in this book adds complexity. A lock means coordination overhead. A retry means you need to think about idempotency. A

rate limiter means you need to choose the right scope and window. These are tools for specific problems, not defaults.

The developers we respect most aren't the ones who use the most patterns. They're the ones who know the patterns well enough to recognize when *not* to use them.

So here's our actual advice: go back to your codebase and look at it with fresh eyes. You'll see the patterns now. Some of them will explain things that confused you before. Some will point to bugs you didn't know you had. And in a few places, you'll realize the pattern was the wrong choice, and a simpler solution was sitting there all along.

That's the real skill. Not just seeing the patterns, but knowing when the code is better without them.

Now go write the code your future self will thank you for.

## About the Authors

**Mohammad Emran Hasan** has spent 25+ years building production software, working across PHP, Laravel, and Symfony. A Laravel framework contributor and past Laracon speaker, he has led engineering teams across both consumer and enterprise systems.

He is the co-founder and CEO of *Klasio* and *FigLab*, where his work centers on AI-assisted engineering. He also teaches *Agentic Engineering*, a cohort course on building production software with Claude Code.

Find him at [@phpfour](#) or [emranhasan.com](#).

**Ahmed Shamim Hasan Shaon** has around 10 years of industry experience spanning engineering management and large-scale system design. A Laravel core contributor, he has stayed close to the developer community, contributing to open source and mentoring engineers through ADPList.

He is a Senior Software Engineer at *Kahf*, working on AI-native development. He also teaches *Think Like a Senior Engineer*, a course on analyzing requirements and the hidden complexity behind features that only look simple.

Find him at [@me\\_shaon](#) or [ahmedshamim.com](#).